

Agentic AI for Information Retrieval

A Multi-Agent System for Multi-Hop Retrieval with
Self-Reflective Validation

Student ID: D03000104

Università degli Studi di Napoli Federico II

Information Retrieval — A.Y. 2024/2025

Prof. Antonio Maria Rinaldi — Dr. Domenico Benfenati

IKNOS-DIETI Lab

September 2, 2026

Contents

1	Introduction to Agentic AI in Information Retrieval	1
1.1	From Reactive Generation to Agency	1
1.2	The Limits of Static Ranking	2
1.3	Problem Statement	3
1.4	Objectives and Contributions	3
1.5	Structure of This Document	4
2	Data and Preprocessing	5
2.1	Requirements for Evaluating an Agentic IR System	5
2.2	On the Datasets Listed in the Assignment Brief	5
2.3	HotpotQA	6
2.4	2WikiMultihopQA	6
2.5	Preprocessing Pipeline	7
2.6	Evaluation Sampling	8
3	Retrieval Methodologies	11
3.1	Classical Foundations	11
3.1.1	Sparse Retrieval: BM25	11
3.1.2	Dense Retrieval	11
3.1.3	Hybrid Fusion and Reranking	12
3.2	Why Single-Shot Retrieval Fails on Multi-Hop Needs	12
3.3	Agentic Retrieval	13
3.3.1	Query Decomposition and Refinement	14
3.3.2	Tool-Augmented Retrieval	14
3.3.3	Knowledge Graph Traversal	15
3.3.4	Self-Reflective Validation	16
3.4	The Role of the Language Model	16
3.5	Architectural Summary	17
4	System Implementation and Evaluation	19
4.1	System Architecture	19
4.1.1	Guarding against non-termination	19
4.2	Implementation Details	20
4.2.1	Indexing	20
4.2.2	Local inference	20
4.2.3	Structured output and its failure mode	20
4.2.4	Budget enforcement	21
4.3	Experimental Setup	21
4.4	Baselines	21
4.5	Evaluation Metrics	22
4.6	Results	22

4.7	Ablation Study	26
4.8	Error Analysis	26
4.9	Discussion	29
5	Conclusions and Future Work	30
5.1	Summary of Contributions	30
5.2	Empirical Findings	30
5.3	Limitations	31
5.4	Future Work	32
5.5	Toward a Thesis Extension	32

Abstract

Most retrieval-augmented systems are pipelines: they retrieve once and generate once. That structure cannot answer a question whose evidence is distributed across documents, because no single document is rankable as the answer. This report presents an agentic alternative — four agents coordinated by an explicit state machine, covering all five tasks of the assignment brief. A Planner decomposes a question into a validated graph of dependent sub-queries; a Retrieval Agent routes each sub-query to BM25, dense, or hybrid search via a seven-rule decision table costing zero model calls; a Knowledge Graph Navigator traverses an entity graph induced from corpus text to recover bridge entities; and a Verifier scores the answer against its own citations using entailment and citation grounding. The central contribution is one backward edge: below its confidence threshold the Verifier returns a re-plan directive to the Planner rather than an answer. That edge is the difference between a pipeline and an agent, and it is isolated by ablation rather than asserted. The system is evaluated on HotpotQA and 2WikiMultihopQA against lexical, dense, hybrid, and agentic baselines, reporting agent-specific cost alongside answer quality. Everything runs locally on one consumer GPU with no paid API.

Chapter 1

Introduction to Agentic AI in Information Retrieval

1.1 From Reactive Generation to Agency

A generative language model is a function from a prompt to a completion. Given an input it emits text and stops. It holds no goal that outlives the call, it cannot observe what its output caused, and it has no mechanism for concluding that what it produced was inadequate. That is an accurate description of a component, and it is also an accurate description of most systems currently labelled retrieval-augmented: a query is embedded, a ranked list is fetched, the list is concatenated into a prompt, and a completion is emitted. Retrieve once, generate once. The behaviour is reactive because nothing in the architecture allows it to be anything else.

Agentic artificial intelligence names the class of systems built to remove that limitation. An agentic system pursues a goal across multiple steps, selects its own actions, invokes external instruments to carry them out, observes what those instruments returned, and revises its behaviour in light of the observation [14, 10]. Four properties are load-bearing, and it is worth separating them because systems are routinely called agentic on the strength of the first three alone.

Autonomy means that the system, not the user, decides what happens next. The user supplies a goal; every intermediate decision — which sub-question to ask, which index to consult, whether the current answer is good enough to return — belongs to the system. **Goal-directed planning** means the system constructs an explicit representation of the work before doing it. A plan is an artefact rather than an emergent side effect: it can be inspected, validated, repaired, compared against a previous plan, and discarded. **Tool integration** means the model is not the answer source but the instrument selector. Facts come from retrieval indexes, graph traversals, and arithmetic; the model’s contribution is deciding which instrument fits which sub-goal and how the outputs compose. **Adaptive reasoning** means the plan is revisable after execution has begun, because an observation can invalidate it.

The fourth property is the difficult one, and it is the one most often absent. Autonomy, planning, and tool use can all be present in a system that is nevertheless a fixed pipeline whose stages happen to be language-model calls: work flows forward, each stage runs exactly once, and nothing discovered late can change anything decided early. The industry touchstones the assignment brief cites — AutoGPT and Microsoft’s Jarvis — are notable precisely for attempting the fourth property, letting a model re-enter its own planning stage on the basis of what its tools returned. The research literature shows the same trajectory. PaperQA retrieves across full-text scientific articles, judges the relevance of what it found, and synthesises an answer from the surviving evidence [3]. The Agentic Reasoning framework adds tool-using sub-agents and a mind-map agent that maintains a structured record of established relations [10]. CollEX supports curiosity-driven multimodal exploration of scientific collections [7], agentic contextual retrieval in telecommunications combines multi-source retrieval with self-reflective validation for network

decision-making [13], and multi-agent geospatial copilots decompose remote-sensing workflows across specialised agents [4]. Chapter 3 examines these approaches in detail. What matters here is the shape they share and the question they leave open: whether the feedback edge, once actually implemented, earns the compute it consumes.

1.2 The Limits of Static Ranking

Classical information retrieval is the study of a scoring function. Given a query and a corpus, rank the documents by estimated relevance and return the top k . The probabilistic relevance framework and its BM25 instantiation formalise this for lexical matching [6]; dense bi-encoders replace term overlap with proximity in a learned embedding space [11]; reciprocal rank fusion combines rankings from several such systems without requiring their scores to be commensurable [1]; and heterogeneous benchmarks such as BEIR measure how well any of it transfers to unseen domains [8]. These are genuine and cumulative advances. Every one of them improves the *ordering of a single list produced from a single query*.

Consider a question from the HotpotQA development set [12], which this report reuses as a running example:

When did the tour begin for the album Taylor Swift released on October 22, 2012?

The corpus contains the two passages that settle it. The passage titled **Red (Taylor Swift album)** states that *Red* is the fourth studio album by Taylor Swift and was released on October 22, 2012. The passage titled **The Red Tour** states that the tour was launched in support of that album and began on March 13, 2013 in Omaha, Nebraska. The gold answer is *March 13, 2013*.

Of the 66,581 passages in the processed HotpotQA corpus, exactly zero contain both *October 22, 2012* and *March 13, 2013*. This is not a statement about ranking quality. It is a statement about the corpus: there is no document for a ranker to place first, because no single document answers the question. A single-shot retriever is structurally incapable here, and so is every refinement layered on top of it. A cross-encoder reranker permutes a candidate set; it cannot insert into that set a passage whose relevance depends on a fact the query does not contain.

The lexical structure of the example sharpens the point. The bridge entity — the album title *Red*, the one term connecting the question to the answer-bearing page — does not occur in the question at all. It has to be *derived* before it can be searched for. A lexical retriever cannot match a term that was never typed, and a dense retriever embeds the question as a whole, so the sub-need “identify the album” is diluted by the sub-need “find its tour date” rather than issued as a query in its own right. The information need is a composition of two atomic needs, and single-shot retrieval resolves compositions only in the fortunate case where some passage already states the composition. This is the compositionality gap that Press et al. measure directly [5], and it is what motivates interleaving retrieval with reasoning steps rather than separating them [9].

Scale turns the difficulty into an impossibility. The two corpora used in this work hold 66,581 and 54,957 passages; the number of ordered passage *pairs* is on the order of 10^9 in each case. Materialising the compositions in advance is not an option, so the composition must be inferred from the question, which is a reasoning operation rather than a ranking one. Two further pressures compound it. The HotpotQA distractor setting populates each question’s candidate pool with passages deliberately chosen to be topically adjacent to the gold ones, so surface similarity is actively adversarial. And relevance for the first hop is defined only relative to an intermediate goal the user never stated: **Red (Taylor Swift album)** is relevant not because it resembles the question but because it supplies the term the second query needs. No relevance judgement over the original query alone can express that.

1.3 Problem Statement

The problem this project addresses is the design of an information retrieval agent that, given a natural-language question and a static corpus, does four things a ranker does not. It must *interpret the intent* behind a question whose surface form under-specifies the retrieval work required. It must *decompose* that intent into an ordered set of sub-goals with explicit dependencies, so that the result of one retrieval can be substituted into the query of the next. It must *orchestrate* the resulting multi-step workflow, choosing an appropriate retrieval instrument for each sub-goal and, where the corpus admits a structured view, traversing that structure rather than searching it. And it must *judge its own answer* against the evidence it actually retrieved, before returning anything.

The fourth requirement is where a pipeline and an agent part company, and it is the central claim this report exists to test. The system described here is, with one exception, a forward pipeline: plan, execute, aggregate, synthesise. The exception is a single backward edge. A Verifier scores each candidate answer against the evidence cited in support of it, blending natural-language-inference entailment, citation grounding, and the agreement between retrieved passages and the answer into one confidence value. When that confidence falls below the configured threshold, the Verifier does not answer. It emits a **ReplanDirective** back to the Planner carrying what could not be established, which sub-queries returned nothing useful, and — the field that makes the loop converge rather than spin — the normalised text of every sub-query already attempted, which the next plan is forbidden to repeat. One edge is a modest architectural change and a categorical behavioural one: it is the only mechanism in the system by which a late observation can revise an early decision.

The system operates under a deliberate constraint. It runs entirely on local hardware: inference is served by **qwen3:8b** at Q4_K_M quantisation through Ollama on a single RTX 5060 with 8 GB of VRAM, measured at 52 tokens per second warm, and no component holds an API key. The constraint sharpens the research question rather than weakening it. At that throughput a schema-constrained model call costs several seconds of wall clock, so every call must be justified against a deterministic alternative. The resulting design treats “where should an agent reason?” as a budgeting problem: tool selection is decided by a seven-rule decision table over cheap lexical features and costs zero model calls, entity linking is alias matching, graph traversal is bidirectional breadth-first search, and the confidence blend is arithmetic. The worst case for a full agentic question is 15 model calls against a hard cap of 20, and the typical cost is four to seven. A system that cannot afford to think everywhere is forced to be explicit about where thinking pays.

1.4 Objectives and Contributions

The objective is a working agentic retrieval system that covers all five tasks of Table 1 of the assignment brief and whose central architectural claim is tested rather than asserted. The contributions are four.

First, a four-agent architecture with full task coverage. A Planner performs autonomous query refinement and multi-step retrieval planning, emitting a validated directed acyclic graph of sub-queries linked by inter-node placeholders. A Retrieval Agent performs tool-augmented retrieval, selecting among BM25, dense, and hybrid search under a conditional reranking gate. A Knowledge Graph Navigator performs graph traversal over an entity graph built from corpus text alone, using bidirectional search to surface the bridge entity that multi-hop questions turn on. A Verifier performs self-reflective validation and fact checking. An orchestrator, specified as an explicit state machine rather than as implicit control flow, coordinates the four.

Second, the Verifier-to-Planner feedback loop, isolated by ablation. The loop is

the difference between this system and the retrieve-once pipeline it is measured against, so it is evaluated as a treatment rather than described as a feature. An `agentic_no_verifier` configuration disables the backward edge while holding everything else fixed, and the trace records which cycle produced the answer that was ultimately selected, so the question “did re-planning actually help?” is answerable from the logs rather than inferred from an aggregate.

Third, an evaluation protocol that prices agency. Answer quality and retrieval quality are reported alongside agent-specific cost — model calls per question, calls avoided by deterministic rules, tool invocations, plan depth, re-plan rate, and latency — so that no accuracy gain is presented without the compute it consumed. Nine configurations, spanning lexical, dense, hybrid, naive-RAG and self-ask baselines as well as the agentic system and its ablations, are run across two multi-hop benchmarks.

Fourth, a fully local and reproducible implementation. Every model runs on-device, prompts are versioned files whose rendered SHA-1 is logged with each call, every sort carries an explicit tiebreaker, and a test asserts that no module imports a network client other than the local Ollama endpoint. The system can be re-run by a reader with a consumer GPU and no budget.

A fifth outcome is not a designed contribution but is reported because it is useful. Schema-constrained planning with an 8B model proved more reliable than expected on structure and less reliable than expected on self-description: across the planning calls measured during development the model produced correctly shaped decompositions in 8 of 8 cases with zero format retries, while mislabelling its own plan’s `strategy` field in 7 of those 8, including obvious bridge and comparison cases. The system therefore derives the strategy from the shape of the validated plan graph and overwrites whatever the model claimed. Where a small model’s self-report can and cannot be trusted is a practical result for anyone building agents at this scale.

1.5 Structure of This Document

Chapter 2 covers data and preprocessing. It explains why the pretraining corpora listed in the brief’s Table 2 cannot support an evaluation requiring relevance judgements, substitutes two multi-hop retrieval benchmarks that carry the necessary ground truth, and documents the passage segmentation, deduplication, and stratified sampling that produce the evaluation and calibration splits.

Chapter 3 surveys retrieval methodologies. It develops the classical foundations — sparse, dense, and fused retrieval with reranking — establishes why single-shot retrieval fails on compound needs, and then examines the agentic alternatives corresponding to each of the brief’s five tasks, locating this system’s design choices within that space.

Chapter 4 presents the implementation and the evaluation. It specifies the orchestrator state machine, the per-agent contracts, the indexing and budget-enforcement machinery, the baseline and ablation configurations, and the metrics; it then reports results, ablations, and a qualitative error analysis.

Chapter 5 assesses what the evidence supports, states the limitations of the study without softening them, and sets out the work that follows from it.

Chapter 2

Data and Preprocessing

2.1 Requirements for Evaluating an Agentic IR System

A dataset is not chosen because it is popular; it is chosen because it can answer the questions a report has to ask. This system’s central claim — that a validation agent can send useful feedback to a planning agent — is only meaningful if it can be falsified with numbers. Four requirements follow, and they are stated before any dataset is named so that the selection reads as derived rather than preferred.

(a) Queries with relevance judgments. Chapter 4 is required to report relevance metrics against baseline IR systems. Recall@ k , nDCG@10 and MRR are all defined over a mapping from queries to relevant documents. A corpus without such a mapping — a *qrels* file, in TREC terminology — makes every one of those metrics undefined, not merely inconvenient to compute. This requirement is therefore non-negotiable, and it eliminates more candidate resources than the other three combined.

(b) Genuinely compositional information needs. The Planner exists to decompose a question into a dependency graph of sub-questions. On a corpus of single-fact look-ups the correct decomposition is the identity, every plan has one node, and the entire agentic apparatus collapses onto the hybrid baseline it is supposed to beat. Measuring a planner requires questions whose answers are not contained in any single document.

(c) Entity and structure annotations. The KG Navigator traverses an entity graph, and a traversal that cannot be scored is a description rather than a result. Sentence-level supporting-fact annotations make retrieved evidence checkable at the granularity the system actually emits; gold relational triples make the traversal itself checkable.

(d) A corpus that indexes and runs on consumer hardware. Every component of this system — generation, embedding, reranking, entailment — runs locally on a single 8 GB laptop GPU, with no paid API anywhere in the project. A web-scale corpus is not merely expensive under that constraint; it is unusable, and a resource that cannot be indexed cannot be evaluated.

BEIR [8] is the natural first place to look, and it satisfies (a) and (d) comfortably across eighteen heterogeneous retrieval tasks. Most of its tasks, however, express single-hop needs and carry no reasoning-structure annotations, so they satisfy neither (b) nor (c). The two datasets adopted below satisfy all four, and the first of them is itself one of BEIR’s tasks, which keeps this choice inside the standard IR evaluation canon rather than outside it.

2.2 On the Datasets Listed in the Assignment Brief

Table 2 of the assignment brief lists five resources: The Pile, Common Crawl, WikiText, OpenWebText and LAION-5B. All five are pretraining corpora. They supply unlabelled text — or, for LAION-5B, weakly labelled image–text pairs — at a scale suitable for fitting the parameters of a language or multimodal model. None of them ships a query set, a relevance judgment, or a gold

answer, because none of them was assembled for that purpose. Requirement (a) therefore fails outright: with no query-to-document relevance mapping there is no $\text{Recall}@k$ and no $\text{nDCG}@10$ to report, and the comparison against baseline IR systems that Chapter 4 is required to present is not computable at all. This is not a deficiency in those corpora. It is a statement about what they are for.

They are, in fact, exactly the right resources for a different layer of this system. The reasoning component used throughout this project is a locally served 8B-parameter language model, and its parametric knowledge — its grasp of English syntax, of world entities, of the shape of a well-formed JSON object — is the product of pretraining on corpora of precisely the kind Table 2 enumerates. The brief’s list describes the substrate the agent brings to the task; it does not describe the benchmark against which retrieval is scored. Those are two different roles, and conflating them is what makes the substitution below look like a departure when it is not one. The brief’s own instruction to “always check for eventual updates of links and datasets via a web search” is the licence to select evaluation data on evaluation criteria, and that is what Sections 2.3 and 2.4 do.

2.3 HotpotQA

HotpotQA [12] is a multi-hop question answering dataset built over English Wikipedia, in which each question is written so that answering it requires combining facts from two distinct paragraphs. This work uses the **distractor** setting, in which every question is accompanied by its two gold paragraphs and eight retrieved distractors, and the **validation** split, which contains 7,405 questions.

Its distinguishing property, and the reason it is the primary dataset here, is the *supporting facts* annotation: alongside the answer string, every question carries a list of (**title**, **sent_id**) pairs naming the individual sentences a human annotator used. This is retrieval ground truth at sentence granularity rather than merely answer ground truth, and it is what makes the supporting-fact metrics **sp_em** and **sp_f1** computable at all. It also fixes the granularity of the rest of the system: evidence is carried as sentences, not passages, so that what the Verifier reasons over and what the metrics score are the same objects.

Reading the distractor contexts of all 7,405 validation questions yields 73,700 paragraphs, which reduce to 66,581 distinct passages after the deduplication described in Section 2.5; 7,119 of the discarded paragraphs were byte-identical repeats, and no title in this split carries two different body texts. The corpus contains 269,602 sentences, a mean of 4.049 sentences and 531.9 characters per passage. Of the 18,005 gold supporting facts, 18,004 resolve against the built corpus. The single exception is a genuine upstream annotation error rather than a preprocessing artefact: question **5ae61bfd5542992663a4f261** cites sentence 902 of the five-sentence paragraph “Jimmy Butler (basketball)”. At a rate of 5.6×10^{-5} this is noise, but it is reported rather than silently dropped, because the same counter is the tripwire that would catch a genuine sentence-identifier drift.

One measured property of this split governs the sampling design in Section 2.6 and is easy to get wrong. HotpotQA labels questions **easy**, **medium** or **hard**, but **all 7,405 validation questions carry level=hard** — the easier levels exist only in the training split. The informative partition here is the reasoning type: 5,918 **bridge** questions, in which one entity must be resolved before the second hop can even be posed, and 1,487 **comparison** questions, in which two independent entities are retrieved and an attribute of each is compared.

2.4 2WikiMultihopQA

2WikiMultihopQA [2] was constructed to expose the reasoning steps a multi-hop question requires, rather than only its endpoints. Its validation split contains 12,576 questions whose 125,760

context paragraphs reduce to 54,957 distinct passages and 202,759 sentences, a mean of 3.689 sentences and 427.2 characters per passage. Like HotpotQA it ships sentence-level supporting facts — 30,687 of them, all of which resolve.

The reason it is in this report is what it ships *in addition*: gold Wikidata evidence triples, 31,120 of them, covering all 12,576 questions. These give the reasoning chain as structured relations rather than as prose, and they convert the KG Navigator from a component this report describes into a component this report measures. They also introduce the most severe leakage risk in the project. Building the entity graph from those triples would let the KG Navigator read the answer key, and every graph-attributed gain in Chapter 4 would be an artefact of that leak. The graph is therefore built from corpus passage text only (Section 2.5); the gold triples are loaded exclusively by the scoring code, and a test asserts that the graph builder never opens a file whose name contains `evidence` or `triple`.

Unlike HotpotQA, all four of 2Wiki’s reasoning types are populated: `compositional` (5,236), `comparison` (3,040), `bridge_comparison` (2,751) and `inference` (1,549). The third of these is the hardest case in either dataset, since it requires both a bridge resolution and a comparison over its results.

This split is also textually messier than HotpotQA, in a way that bears directly on ground truth. Because the same article appears in the contexts of many questions, and because those contexts were detokenised inconsistently, 1,720 of the 54,957 titles ship two or more variants of the same article (“Feo(c. 1471” versus “Feo (c. 1471”), producing 2,617 title collisions. Since `sent_id` indexes into a sentence list, the variant retained decides whether a gold fact lands on the right sentence. The resolution rule and its measured cost are given in Section 2.5.

2.5 Preprocessing Pipeline

The pipeline is deliberately thin. Every transformation it applies is one that cannot be undone downstream, and every transformation it declines to apply is one a consumer can perform for itself.

Passage segmentation. Both datasets already ship each context paragraph as a list of sentences, and that list is adopted verbatim. Nothing in this project calls a sentence splitter — not NLTK, not PySBD, not a regular expression. The reason is that supporting facts are (`title`, `sent_id`) pairs, so the sentence list *is* the identifier space of the ground truth. Re-splitting anywhere downstream would renumber every gold fact while `sp_em` and `sp_f1` continued to produce entirely plausible numbers — a silent failure, which is the worst kind. The split is performed once, at corpus build time, and persisted. Empty sentences keep their slot, because the slot is the identifier.

Canonical identifiers. Each passage is keyed by `doc_id = {source}:{quote(title)}`, for example `hotpotqa:Accolade%20Wines`. The identifier is URL-safe, stable across rebuilds, and derived from the title alone, so that a passage returned by any of the three scorers, an entity node in the graph, and a gold supporting fact all name the same object in the same way.

Deduplication. The distractor setting repeats paragraphs across questions, so the raw paragraph stream is roughly an order of magnitude larger than the distinct corpus. HotpotQA collapses cleanly: 7,119 byte-identical repeats removed, zero collisions. 2Wiki removes 68,186 identical repeats but leaves 2,617 cases in which one title carries two different texts. The rule is to keep the variant with the *most sentences* — the widest identifier space — with ties going to the first occurrence. The rule is decided without consulting any gold data, and its cost is measured: over 2Wiki’s 30,687 gold facts, keeping the first occurrence would leave 70 identifiers out of range and 211 pointing at the wrong sentence, whereas keeping the widest leaves 0 out of

range and 150, or 0.489%, landing on a slightly different detokenisation of the intended sentence. Both residuals are counted in the build statistics rather than rounded away.

Normalisation, and what is deliberately not done. The only transformation applied to corpus text is the collapse of whitespace runs inside a sentence, which is necessary because HotpotQA prefixes continuation sentences with a space and 2Wiki does not. There is no lowercasing, no stopword removal, no stemming and no punctuation stripping at the corpus level. Those are properties of a *scorer’s analyser*, not of a corpus: BM25 applies Snowball stemming and an English stopword list inside its own analyser, whereas the dense encoder needs original case and punctuation to distinguish proper nouns, and the NLI premise encoder needs well-formed sentences. Baking any of those steps into the stored text would be irreversible and would degrade two of the three scorers in order to help one. Conversely, one convention *is* shared across all of them: every scorer sees the same title-prefixed unit, "**Title. body**". A channel that indexes titles, fused with a channel that does not, produces rankings that are not comparable, and rank fusion would absorb the mismatch silently rather than expose it.

Entity extraction and alias resolution. Graph nodes are passage titles, normalised by NFKD decomposition, lowercasing, diacritic stripping, removal of parenthetical disambiguators and whitespace collapse. A node’s alias set is its surface title plus the no-parenthetical variant plus the comma-inverted variant, and the aliases are compiled into a trie for longest-match linking at query time. No named-entity recogniser and no language model is involved, so linking is deterministic and costs nothing at run time.

Entity graph construction. A directed edge $A \rightarrow B$ is created when an alias of B occurs in the text of passage A . Each edge stores the `doc_id`, the `sent_id` and the containing sentence, which is what makes a traversal result citable evidence at the same granularity as a retrieved passage rather than an unscorable decoration. The relation label is the shortest verb-anchored token window between the two mentions, truncated to six tokens and defaulting to `mentions`; there is no parser dependency and no generation step. Edge weight is $1/(1 + \log(1 + \text{outdeg}(A)))$, so hub pages contribute weak edges. As recorded in `data/indexes/{dataset}_kg_meta.json`, the HotpotQA graph retains 64,733 nodes and 123,759 edges from 417,881 candidates and builds in 49.4s; the 2Wiki graph retains 52,959 nodes and 63,073 edges from 161,572 candidates and builds in 32.9s. Pruning is dominated by a document-frequency cap on mentions, which removed 446 and 172 hub nodes respectively: strings such as `one` and `united states` are legitimate page titles, but their occurrences are not relational assertions, and without the cap the graph collapses into a hairball centred on them. Verb-anchored relation labels are recovered for 76.8% of HotpotQA edges and 79.0% of 2Wiki edges, and Section 3.3.3 is explicit about what the remainder costs.

Indexing. Each corpus is indexed twice: a BM25 index with Snowball stemming, and an exact inner-product FAISS index over 384-dimensional `bge-small-en-v1.5` embeddings. Together these occupy 129.1 MB for HotpotQA and 103.6 MB for 2Wiki — small enough to hold in memory, which is why exact search is used in place of an approximate index and why no recall loss is introduced at the index level. The 2Wiki build took 4.1s for BM25 and 97.6s for the dense index on GPU.

2.6 Evaluation Sampling

Evaluation is run on a frozen 250-question sample per dataset rather than on the full split, and the reason is arithmetic. Local generation runs at a measured 52 tokens per second warm on an RTX 5060 with 8 GB of VRAM, giving a mean schema-constrained planner call of about 3.75s

and a mean agentic question cost of roughly 22–25 s once retrieval, reranking and entailment are included. One agentic configuration over HotpotQA’s full 7,405-question validation split is on the order of 45 hours, and the evaluation grid comprises nine configurations across two datasets. At 250 questions the same grid is approximately 12–16 hours, which is feasible on this hardware; at 7,405 it is not.

The sample is drawn once, by `scripts/sample_eval_set.py` with seed 42, materialised to `data/processed/{dataset}_eval_250.jsonl`, and never re-sampled at run time. Its SHA-256 is recorded in every run’s `meta.json`, so a silently changed sample is detectable after the fact.

The stratification variable is question type, not difficulty, and that is a measured decision rather than a stylistic one. As reported in Section 2.3, all 7,405 HotpotQA validation questions are labelled **hard**. Stratifying on `level` would therefore produce a single stratum — an unstratified sample presented as a stratified one, with the label doing no work while implying that it does. Stratifying on `type` preserves the 0.799/0.201 bridge-comparison ratio exactly, at 200 and 50 questions. For 2Wiki the four reasoning types are preserved at 104 compositional, 60 comparison, 55 bridge-comparison and 31 inference. Table 2.2 gives the full composition against the split.

Two hundred and fifty questions is enough to separate the systems compared here. In the worst case — a metric near 0.5, treated as an unpaired binomial proportion — the 95% normal-approximation half-width is $1.96\sqrt{0.25/250} \approx 0.062$, an interval of roughly ± 6.2 points. The comparisons that matter in Chapter 4 are paired, however: every configuration answers the identical 250 questions, so the bootstrap is taken over per-question *differences* (1,000 resamples, `evaluation.bootstrap_samples`), and the error correlation between systems sharing a corpus, an index and a question set makes those intervals materially tighter than the unpaired figure suggests; the realised widths are reported with the results rather than assumed here. The three component ablations run on a fixed 150-question subsample of the same 250 to keep the grid inside its time budget, and the results table says so in its caption.

Each dataset also gets a 50-question calibration slice, `{dataset}_calib_50.jsonl`, written beside the evaluation file in `data/processed/` and verified to have zero question-identifier overlap with the evaluation 250. Its sole purpose is to fix the Verifier’s `confidence_threshold` — currently 0.55 — before the evaluation run begins. That threshold controls the re-plan rate, which is itself one of the headline agent-specific measures, so sweeping it on the evaluation set would be tuning on the test set and would invalidate the very metric it was tuned to improve.

Table 2.1: Corpus statistics after preprocessing, read from `{dataset}_corpus_stats.json`. Passages are de-duplicated by content under a canonical title-derived id; sentence splits are the datasets’ own and are persisted verbatim, because gold supporting facts index into them by position and re-splitting anywhere downstream would renumber the ground truth while every metric still looked plausible. - marks a statistic the build did not record.

Statistic	hotpotqa	twowiki
Questions in split	7 405	12 576
Paragraphs read	73 700	125 760
Passages after de-duplication	66 581	54 957
Identical paragraphs dropped	7 119	68 186
Title collisions resolved	0	2 617
Sentences	269 602	202 759
Mean sentences per passage	4.05	3.69
Mean passage length (chars)	531.9	427.2
Mean sentence length (chars)	130.6	115.1
Questions with gold triples	0	12 576
Gold supporting facts	18 005	30 687
Supporting facts unresolvable	1	0
Evaluation sample	250	250

Table 2.2: Composition of the frozen evaluation slice against the full split. The slice is stratified on question type and sampled once, with seed 42, by `scripts/sample_eval_set.py`; its SHA-256 is recorded in every run’s `meta.json` and it is never re-sampled at run time. HotpotQA’s validation split is entirely `level=hard`, so difficulty gives one stratum and a silently unstratified sample; question type is the informative split there.

Question type	Split <i>n</i>	Split share	Sample <i>n</i>	Sample share
<i>hotpotqa</i>				
bridge	5 918	0.799	200	0.800
comparison	1 487	0.201	50	0.200
<i>total</i>	7 405	1.000	250	1.000
<i>twowiki</i>				
bridge_comparison	2 751	0.219	55	0.220
comparison	3 040	0.242	60	0.240
compositional	5 236	0.416	104	0.416
inference	1 549	0.123	31	0.124
<i>total</i>	12 576	1.000	250	1.000

Chapter 3

Retrieval Methodologies

3.1 Classical Foundations

The agentic components described later do not replace classical retrieval; they schedule it. Every sub-query the Planner emits is ultimately answered by one of the three mechanisms below, so their individual blind spots determine what the agent layer has to compensate for.

3.1.1 Sparse Retrieval: BM25

BM25 [6] scores a document d against a query q by summing, over the query terms, a saturating function of term frequency weighted by inverse document frequency:

$$\text{score}(q, d) = \sum_{t \in q} \text{IDF}(t) \cdot \frac{f(t, d) (k_1 + 1)}{f(t, d) + k_1 \left(1 - b + b \frac{|d|}{\text{avgdl}}\right)}.$$

The saturation parameter k_1 controls how quickly repeated occurrences stop adding score; the length normalisation parameter b controls how strongly long documents are penalised. The values used here, $k_1 = 0.9$ and $b = 0.4$ (`retrieval.sparse`), sit below the classical 1.2 and 0.75 deliberately: the indexed unit is a single Wikipedia lead paragraph averaging 4.05 sentences and 531.9 characters, not a full document. At that length repetition carries little extra evidence and length differences are largely incidental, so heavy normalisation would penalise informative paragraphs for being informative.

BM25 matches rare surface forms exactly, needs no training, is interpretable term by term, and answers in milliseconds. It also supplies the analysed lexicon and per-term document frequencies that the routing rules of Section 3.3.2 consume, which nothing else in the system can provide. Its weakness is vocabulary mismatch: a query and a relevant passage sharing no surface term after stemming score zero against one another, however obviously they concern the same thing.

3.1.2 Dense Retrieval

A dense retriever embeds queries and passages independently into a shared vector space and ranks by similarity. This work uses `bge-small-en-v1.5` [11], a 384-dimensional bi-encoder, with vectors L2-normalised so that inner product equals cosine similarity, and an exact FAISS flat inner-product index. Exact rather than approximate search is affordable because the corpora are small — 66,581 and 54,957 passages — and it matters because any recall the system fails to achieve is then attributable to the retrieval method rather than to the data structure. The model is used asymmetrically, as its authors intend: passages are encoded bare, queries are prefixed with `"Represent this sentence for searching relevant passages: "`.

Its strength is paraphrase, exactly the case BM25 cannot see; its weakness is the mirror image. Query and passage never meet, and each is compressed into 384 dimensions before comparison, a bottleneck that preserves topic far better than identity — so two similarly named people or two films sharing a title are liable to be conflated. The two failure modes are close to complementary, which is the standing argument for combining them.

3.1.3 Hybrid Fusion and Reranking

The two ranked lists are combined by Reciprocal Rank Fusion (RRF) [1], which discards the scores and combines positions:

$$\text{RRF}(d) = \sum_{r \in R} \frac{1}{k + \text{rank}_r(d)}, \quad k = 60,$$

with k from `retrieval.hybrid.rrf_k`. Rank-based fusion is preferred over score normalisation because BM25 scores are unbounded and depend on corpus statistics and query length, while cosine similarities from a normalised encoder occupy a narrow band near the top of $[-1, 1]$: the two share no scale and no principled conversion. Normalisation manufactures one, but the fused score then depends on the shape of the candidate list, so a single outlier at the tail can reorder the head. Ranks are ordinal and scale-free, and the ordinal information is the part on which the two retrievers can meaningfully be said to agree.

Fusion improves recall but leaves head precision largely to chance, which is what reranking addresses. A cross-encoder — `ms-marco-MiniLM-L-6-v2` here — concatenates query and candidate and runs full self-attention over the pair, so their terms interact directly. It is more expressive than a bi-encoder for the same reason it is more expensive: nothing about the passage can be precomputed, so scoring n candidates costs n forward passes and admits no index. That cost fixes the depth parameters, `top_n` = 50 before reranking and `top_k` = 10 after. Reranking is also often decorative, so when the fused top-1 already leads the runner-up by a relative RRF margin of at least `rerank_margin_gate` = 0.15 the fifty forward passes cannot change the head of the ranking and are skipped — a saving counted in the trace rather than assumed.

3.2 Why Single-Shot Retrieval Fails on Multi-Hop Needs

Everything in Section 3.1, up to and including the cross-encoder, computes a scoring function $s(q, d)$. The three methods differ enormously in expressive power — from a bag-of-words sum, through a 384-dimensional dot product, to full cross-attention over the pair — but they do not differ in their arguments. All of them assume that the relevance of a document is determinable from the query and that document. For a large class of information needs this assumption is correct and unremarkable. For a bridge question it is false, and the consequence is not that these systems perform poorly but that they are solving the wrong problem.

A worked example. Consider question `5a7793275542992a6e59df04`, drawn from the HotpotQA evaluation sample of Section 2.6:

Which country is the firm that owns Babycham located?

The gold answer is *Australia*, and the two gold supporting facts are the opening sentences of two passages:

Babycham, sentence 0 — “Babycham is the trade name of a light, sparkling perry invented by Francis Showering, a brewer in Shepton Mallet in Somerset, England; the name is now owned by Accolade Wines.”

Accolade Wines, sentence 0 — “Accolade Wines is a major global wine business with headquarters in South Australia.”

The first hop is easy, and easy in a way that flatters BM25 specifically: “Babycham” is a near-unique term in a 66,581-passage corpus, so lexical matching on that term alone puts the correct passage at rank one. This is precisely the situation routing rule R4 of Section 3.3.2 exists to detect.

The second hop is not merely harder. Compare the query terms — *country*, *firm*, *owns*, *Babycham*, *located* — against the second gold sentence. Not one of them occurs in it. BM25 has nothing to match and assigns no score on any discriminative term. A bi-encoder produces a non-zero similarity, but the query embedding is dominated by *Babycham* and a possessive relation, while the passage embedding says *wine business*, *global*, *South Australia*. The passage sits among the corpus’s other company profiles, and nothing in it distinguishes it from any of them with respect to this query.

Why the reranker does not close the gap. There are two reasons, and only the second is fundamental. The first is mechanical: a cross-encoder reorders a pool it is handed, so a passage that never enters the pool cannot be recovered by reordering, and widening the pool postpones the problem rather than solving it.

The second reason survives any pool width and any model capacity. Suppose the *Accolade Wines* passage is placed in the pool by fiat, and suppose the cross-encoder is arbitrarily large and perfectly trained. It is presented with the pair (question, passage) and asked to judge relevance. The information that would justify a high score — that Accolade Wines is the firm that owns Babycham — is present in neither element of that pair. It is asserted in a third document. The relevance of the second-hop passage d_2 therefore does not factorise as a function of the query–document pair; it is a conditional quantity $\text{rel}(q, d_2 \mid d_1)$ depending on a document the ranker was never given. Increasing model capacity changes the function class being searched, not the arguments the function receives, so a better ranker is a better approximation of the wrong quantity. This is why the failure is properly described as a specification error rather than a ranking error, and it is the single fact from which the rest of this chapter follows.

What would work. The query that retrieves *Accolade Wines* trivially is “Where is Accolade Wines located?” — a query with the same rare-key property that made the first hop easy. It cannot be written in advance, because the string “Accolade Wines” appears nowhere in the user’s question; it exists only as the output of the first retrieval. The missing capability is therefore not ranking quality but the ability to issue a second query whose text is conditioned on the result of the first. That is a control structure, not a scoring function, and no single-shot architecture can express it however good its scorer is.

Scope of the argument. This failure mode is not an edge case. Bridge questions are 5,918 of HotpotQA’s 7,405 validation questions, or 79.9%, and 2Wiki’s compositional, inference and bridge-comparison types — 9,536 of 12,576 questions — share the same conditional structure. Comparison questions fail differently, and the distinction is worth stating rather than blurring: there both gold passages are individually retrievable from the original query, since both entities are named in it, so the difficulty is coverage at a fixed cut-off plus the fact that the answer is the output of an *operation* over two passages rather than the content of either. That is a weaker claim, and Section 3.3.1 accordingly gives the two shapes different plan topologies. How far the single-shot baselines fall below the achievable ceiling is a measurement, reported in Chapter 4; the point here is that the ceiling exists by construction and cannot be lifted by tuning.

3.3 Agentic Retrieval

The remedy indicated above is a retrieval process that carries state between rounds. That is the common thread through the agentic IR literature: treating a language model as an autonomous

agent that plans, invokes tools and revises [14, 10], whether the domain is scientific literature [3], multimodal collections [7], telecommunications networks [13] or geospatial workflows [4]. The four agents below cover, between them, the five tasks of the assignment brief’s Table 1.

3.3.1 Query Decomposition and Refinement

The Planner converts a question into a directed acyclic graph of at most five sub-queries. Each node carries its own text, an explicit dependency list, an intent and an expected answer type, and may contain template placeholders of the form `{{q1.answer}}`, `{{q1.entity}}` or `{{q1.title}}`. Execution proceeds in level-synchronous topological order, and a node’s placeholders are substituted from the state of its dependencies immediately before it is routed and retrieved. The second-hop query is therefore a genuinely different string at retrieval time from the one the Planner emitted — exactly the capability Section 3.2 identifies as missing — and because the resolved text differs lexically from the template, routing is recomputed after substitution rather than inherited.

This is the same territory as Self-Ask [5], which interleaves follow-up questions inside a single generation, and IRCOT [9], which interleaves retrieval with chain-of-thought steps. Four differences matter. The plan here is an explicit, validated data structure rather than a decoding trajectory: cycles, dangling dependencies, over-long plans and unbound placeholders are repaired mechanically before execution and every repair is recorded, so a model-generated plan is never executed on trust. It is a graph rather than a chain, so a comparison question yields two independent roots retrieved in the same level where a linear follow-up mechanism must serialise them. Query expansion rides inside the same planner response as a per-node list of at most two rewrites, fused with RRF at retrieval time, so expansion costs *zero* additional model calls. And the plan can be revised on evidence supplied by a downstream agent (Section 3.3.4), which no single-pass decoding trajectory can do.

Two Planner decisions followed from measurement. The plan’s **strategy** label is derived from the shape of the validated graph and overwrites whatever the model reported: on eight representative questions at temperature 0 with a constrained schema, **qwen3:8b** produced structurally correct decompositions 8 times out of 8 but labelled the strategy **single_hop** in 7 of those 8, including unambiguous bridge and comparison cases. It builds the graph well and names it badly, and since that label gates a deterministic comparison shortcut, trusting it would silently disable one; both labels are logged, so their disagreement rate is itself a reportable result about the self-knowledge of a small local model. Separately, the model reliably emits a final *combiner* node restating the question and depending on every other node, whose answer comes from composing its dependencies rather than from any passage; detecting and skipping it removes a third of all retrieval on a three-node comparison plan.

The Planner degrades along a deterministic ladder rather than failing: a comparison template, then a two-node bridge template, then an identity plan routed to hybrid search with reranking. That floor is by construction precisely the **hybrid_rerank** baseline, so the agentic system can never score below its strongest non-agentic competitor for reasons of infrastructure failure — only for reasons of judgement.

3.3.2 Tool-Augmented Retrieval

Tools are declared in a registry as a name, a callable, an argument schema and a natural-language description. Seven are exposed across two families: **bm25_search**, **dense_search**, **hybrid_search** and **rerank** over the indexes, and **kg_link**, **kg_neighbors** and **kg_path** over the entity graph. The same description strings that document a tool to a human are the strings injected into the tool-selection prompt, so model-facing documentation and executable registry cannot drift apart — a small property that removes an entire class of silent bug in which an agent is told about a tool that no longer behaves as described.

Selection among the retrieval tools is a seven-rule decision table over features of the resolved sub-query: token count after stopword removal, presence of a quoted span, number and token share of capitalised runs, out-of-vocabulary rate against the BM25 lexicon, minimum document frequency among content terms, and whether the node has dependencies. The rules encode Section 3.1 directly. A planner hint is honoured (R1); an out-of-vocabulary rate above 0.34 routes to dense search (R2), because BM25 cannot score terms absent from its lexicon; short quoted or proper-noun-dense queries (R3) and queries containing a term of document frequency three or fewer (R4) route to BM25, a very rare term being a near-unique key — the *Babycham* case above; dependent nodes (R5) and long or entity-free queries (R6) route to hybrid, since a bridge-filled query mixes one precise entity with paraphrased relation text and neither channel alone suffices; and a default row (R7) makes the table total.

The table costs zero model calls, and that deserves stating as an argument rather than as an optimisation. An agent that spends a language-model call on a decision a rule already settles is spending the user’s latency budget, not demonstrating intelligence. At the measured 52 tokens per second a schema-constrained call costs roughly 3.75s, whereas the routing rule is a handful of look-ups against an already-loaded lexicon; multiplied across sub-queries, 250 questions, two datasets and nine configurations, that is hours of wall clock purchased for a decision the default row would often have made identically. Autonomy is a property of which decisions a system makes without a human, not of which decisions it routes through a transformer.

The claim is nonetheless empirical and is treated as such. Clearing the retriever’s `heuristic_shortcut` flag routes every sub-query through the selection prompt instead; running that configuration on the disjoint 50-question calibration slice yields the rule–model agreement rate, the change in nDCG@10 and the change in wall clock, and reporting those three numbers is what converts the paragraph above from an assertion into a result.

This is also where reinforcement learning would enter, and why it does not enter here. Tool routing is a policy over a small discrete action space with an observable reward, which is text-book territory for a contextual bandit or a policy-gradient learner. The obstacle is specific and measurable: the reward would have to derive from gold supporting facts, and the only data carrying those labels is the evaluation data, so an on-policy learner would be training on the test set and any gain would be uninterpretable. The one leakage-free signal available is the 50-question calibration slice, far too small to fit a policy that reliably beats a seven-rule table. Reinforcement learning is therefore listed as future work in Chapter 5 rather than claimed here.

3.3.3 Knowledge Graph Traversal

The KG Navigator operates on the entity graph built in Section 2.5. Entity linking is an alias-trie longest match over normalised titles and costs no model call. Traversal is bounded at two hops and twenty-five neighbours per node (`agents.kg.max_hops`, `max_neighbors`) and expanded in deterministic order of descending edge weight, so a repeated run explores the graph in exactly the same sequence.

The highest-value operation is bidirectional breadth-first search: given two or more linked seeds it expands from each and reports the meeting node as the *bridge entity*. Applied to the example of Section 3.2, the alias “Accolade Wines” occurring in the text of the *Babycham* passage is an edge in the graph by construction, so the bridge entity that the retrieval stack could not surface by ranking is recovered by a graph walk in milliseconds, and recovered without a generation step where an equivalent model call would cost seconds. This is the concrete sense in which graph traversal is not decorative: it is a second, structurally different mechanism for the one operation single-shot retrieval provably cannot perform.

Traversal is also the most explainable component, because the path *is* the explanation. Every edge stores the document identifier, the sentence identifier and the sentence asserting it, so a traversal result enters the evidence pool at the same sentence granularity as a retrieved passage, is cited in the same way, and is scored by the same supporting-fact metrics; a graph result that

could not be cited would be unfalsifiable. The limits should be equally clear: the graph is induced from mention co-occurrence, not curated from an ontology, and 23.2% of HotpotQA edges and 21.0% of 2Wiki edges carry no label beyond **mentions**, so relation semantics are weak and the structure is best understood as a navigation aid rather than a knowledge base. Comparison questions are handled accordingly: with two seeds and no path expected, each seed’s attribute-bearing sentences are returned instead of forcing a spurious connection, and an empty result is a legal outcome.

3.3.4 Self-Reflective Validation

The Verifier receives the Synthesizer’s answer with the evidence pool and decides whether it is supported. Cited identifiers that do not exist are recorded as hallucinated citations and contribute no support, which alone catches a common failure of small local models. A natural language inference model, **cross-encoder/nli-deberta-v3-base**, is then run with each cited sentence as premise and the answer sentence as hypothesis, and the maximum probability of entailment becomes the NLI support; the Synthesizer emits that answer sentence as a separate field precisely so the hypothesis is a well-formed declarative sentence, which is what an NLI model is trained on, rather than the usual concatenation of question and answer. Two further signals are computed — citation grounding, the fraction of claims with at least one valid supporting citation, and retrieval agreement, the fraction of executed sub-queries whose top-ranked title appears among the cited evidence — and the four combine as

$$\text{conf} = 0.45 \text{ nli} + 0.25 \text{ citation} + 0.15 \text{ retrieval} + 0.15 \text{ llm},$$

with weights from **agents.verifier.weights**, renormalised when the model term is absent.

That last term is gated: the adjudication call is issued only when the confidence computed without it falls inside $[0.40, 0.70]$, the threshold of 0.55 plus or minus an uncertainty band of 0.15, because confident acceptances and confident rejections cannot have their verdict changed by it. Above the threshold the verdict is **accept**. Below it, with re-plans remaining, the verdict is **revise** and the Verifier emits a re-plan directive carrying what could not be established, which sub-queries returned nothing useful, which entities are already covered, and — the field that makes the loop work at all — the normalised texts of every sub-query already tried. Without that ban list, an 8B model told only that its previous attempt was insufficient reliably re-emits the same decomposition and burns both re-plans for nothing. Below the threshold with no re-plans left the verdict is **abstain**, a label rather than a refusal: the best candidate is still returned, so answer metrics remain computable.

A verifier can be wrong in two directions and the design has to survive both. Too strict, it triggers re-planning that costs latency and risks replacing a good answer with a worse one; too lenient, it rubber-stamps errors while reporting high confidence. Answer selection at termination is therefore the argument maximising confidence over *all* cycles, ties broken toward the earlier and cheaper answer, so a bad re-plan cannot silently overwrite a good first attempt; and the error taxonomy of Chapter 4 contains explicit **verifier_false_accept** and **verifier_false_reject** categories, computed post hoc against gold supporting facts, so the loop is falsifiable rather than merely asserted. The threshold itself is swept on the disjoint calibration slice of Section 2.6 and frozen before the evaluation run, because it controls the re-plan rate and tuning it on the evaluation set would corrupt one of the headline agent metrics.

3.4 The Role of the Language Model

The language model is the component that generalises, and everything else in this system exists to keep the number of calls small and their inputs short. Three decisions genuinely require it: mapping an unseen compound question onto a dependency structure, which is open-ended and

cannot be enumerated by rules; composing a short answer span from heterogeneous evidence sentences, which requires reading; and adjudicating the narrow band where the numeric confidence signals disagree, which requires judgement arithmetic does not supply. Everything else is deterministic — tool routing, entity linking, graph traversal, entailment, evidence deduplication and ranking, comparison arithmetic over parsed dates and numbers, and query expansion, which rides inside the planner call. Each is a decision over a small closed action space with a cheap and reliable feature, where a model call would add latency and a new failure mode, malformed output, without adding accuracy. The worst case over a whole question is therefore 15 calls against a cap of 20, with 4–7 typical, and two are reserved so synthesis and verification always run even when the Planner has exhausted the budget.

Serving that model locally shaped the design more than any other single factor, and Chapter 4 gives the full configuration. Two consequences belong here because they changed the method rather than the deployment. Throughput of 52 tokens per second makes prompt length a latency cost, which is why the re-plan prompt passes retrieved *titles* rather than retrieved text, holding it to a few hundred tokens instead of several thousand. And **qwen3** is a hybrid reasoning model that emits a `<think>` block by default, whose measured effect is worse than added latency: with thinking enabled and a 128-token prediction budget, the model spent the *entire* budget on a 503-character reasoning block and produced no answer at all. Thinking does not merely delay the response, it starves it, so suppression is applied three ways and whatever leaks through is logged. Structured output is parsed down a five-rung ladder ending in a repair prompt, and parse failures are counted per agent — because if the Planner’s rate is high, what the results measure is partly the fallback ladder rather than the model, and the report has to say so.

3.5 Architectural Summary

Figure 3.1 shows the resulting system. Read left to right it is a pipeline: a Planner decomposes, a Retrieval Agent routes each sub-query to the indexes, a KG Navigator recovers bridge entities from the entity graph, a Synthesizer composes a cited answer, and a Verifier scores it. The edge that makes it an agent rather than a pipeline is the dashed one drawn above: below a confidence of 0.55 the Verifier returns a re-plan directive to the Planner instead of returning an answer, and the system tries again with an explicit record of what it already knows and what it must not repeat. Every other component in this chapter is standard IR machinery arranged carefully; that backward edge is the contribution, and Chapter 4 isolates its effect by ablation rather than assuming it.

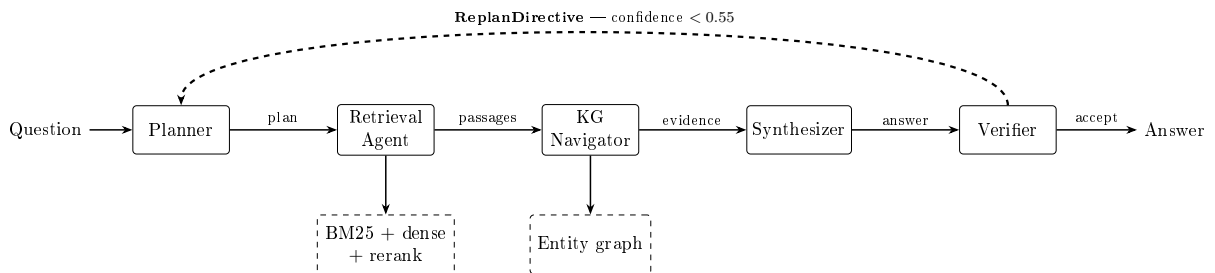


Figure 3.1: The four-agent architecture. Solid arrows are the forward retrieval path; dashed boxes are offline artefacts built in Section 2.5. The dashed backward edge is the Verifier-to-Planner feedback loop — the only information that flows against the pipeline, and what distinguishes this system from a retrieval-augmented generation pipeline.

Mapped onto the assignment brief’s Table 1, the Planner covers autonomous query refinement and multi-step retrieval planning, the tool registry and routing table cover tool-augmented retrieval, the KG Navigator covers knowledge graph traversal, and the Verifier covers self-reflective

validation and fact checking. Five tasks are addressed by four agents because refinement and planning are the same act performed at different points in the loop: once before evidence exists, and again after the Verifier has said what is missing.

Chapter 4

System Implementation and Evaluation

Chapter 3 argued that a multi-hop information need cannot be served by a single retrieval round, and set out the four agents that answer it. This chapter describes how those agents are built, how the system is measured, and what the measurements show. It closes with an error analysis, because a system that is right for the wrong reasons is not a result.

4.1 System Architecture

The brief names four modules — a query planner, a tool-invocation manager, a retrieval engine and a validation component. They map onto the four agents of Chapter 3 and an orchestrator that sequences them.

The orchestrator is an *explicit* state machine rather than a chain of function calls. Nine states (`INIT`, `PLAN`, `EXECUTE`, `AGGREGATE`, `SYNTHESIZE`, `VERIFY`, `REPLAN_GATE`, `FINALIZE`, `DONE`) and fourteen labelled transitions are enumerated in a table in the source, and the transition sequence taken by each question is written into its trace. This costs more code than implicit control flow and buys two things. First, the mechanism is inspectable: the claim that the system re-plans is checkable against a recorded path rather than asserted. Second, every exit is named, so a question that terminates early is attributable to a specific guard rather than disappearing into a generic failure count.

Only one transition runs backwards. `REPLAN_GATE` returns to `PLAN` when the Verifier’s confidence falls below threshold, carrying a `ReplanDirective`. Removing that single edge reduces the system to a retrieve-then-generate pipeline, which is why the ablation in §4.7 isolates it directly.

4.1.1 Guarding against non-termination

An agent that can send work backwards can loop. Five guards are evaluated in order at `REPLAN_GATE`, and the first to fail names the termination reason recorded in the trace: the re-plan count must be below `max_replans` (2); the iteration count below `max_iterations` (6); at least three LLM calls must remain; elapsed time must be under 60% of the 300 s wall-clock budget; and the directive must be informative, meaning it names at least one missing item, failed sub-query or suggestion.

A sixth check applies to the plan that comes back. A local 8B model told that its previous attempt was insufficient will frequently re-emit the same decomposition in different words. The system therefore normalises each sub-query — lowercased, punctuation and placeholders stripped, stopwords removed — and rejects a revision whose sub-queries pair with the previous plan’s at a Jaccard similarity of 0.85 or above. Without this, both permitted re-plans can be consumed re-running an identical plan, and the feedback loop becomes pure cost.

4.2 Implementation Details

4.2.1 Indexing

Both corpora are indexed twice. The sparse channel is BM25 over stemmed terms ($k_1 = 0.9$, $b = 0.4$); the dense channel is `bge-small-en-v1.5` [11] embeddings in a FAISS inner-product index over L2-normalised vectors, which makes inner product equivalent to cosine similarity. Both are exact: at this corpus size an approximate index would trade recall for a speed advantage the system does not need.

For 2WikiMultihopQA’s 54,957 passages the sparse index builds in 4.1 s and the dense index in 97.6 s (95.8 s of which is GPU encoding), occupying 103.6 MB on disk against HotpotQA’s 129.1 MB. The asymmetry between the two build times is the reason the encoder is given the GPU here and taken off it later (§4.2.2).

The knowledge graph is induced from passage text alone. For HotpotQA, 417,881 candidate mentions yield 123,759 retained edges over 64,733 nodes in 49.4 s, with 446 hub nodes pruned; for 2WikiMultihopQA, 161,572 candidates give 63,073 edges over 52,959 nodes in 32.9 s with 172 hubs pruned. 76.8% and 79.0% of edges respectively carry a verb-anchored relation label; the remainder are recorded as untyped co-mention, which the traversal still uses for connectivity.

That the graph is built from passage text and never from 2WikiMultihopQA’s gold evidence triples is not a stylistic preference. Those triples are the answer key for the very capability the KG Navigator is being measured on, and a graph built from them would make every KG-attributed gain an artefact. The constraint is enforced by a test that fails if the graph builder opens any file whose name contains `evidence` or `triple`.

4.2.2 Local inference

All generation runs on `qwen3:8b` at Q4_K_M quantisation through Ollama, on an RTX 5060 laptop GPU with 8 GB of VRAM. No component of this project contacts a paid API, and a test enforces that no module outside the model client imports an HTTP library at all.

Measured throughput is 52 tokens per second with the model warm, and a schema-constrained planning call averages 3.75 s. Two properties of the hardware shape the design more than the throughput figure does.

The first is that the card is effectively full. The model and its 8192-token KV cache occupy 6,707 of 8,151 MiB with the desktop’s own allocation included, leaving roughly 1.4 GiB. The three sentence-transformer encoders — query embedding, cross-encoder reranking and NLI — therefore run on CPU at query time, where they cost well under a second per question against a 300 s budget. The GPU is used for encoders only during offline index construction, with the language model unloaded. An encoder co-resident with the model produces an out-of-memory failure partway through an evaluation run, which is the most expensive failure mode available: hours of work lost with partial results.

The second is that the first call after a model load costs approximately 9.5 s in CUDA warm-up, evaluating a 19-token prompt. Warm, the same work takes under two seconds. Latency figures in this chapter are therefore reported from runs with the model resident, and the harness records whether the response cache was cold so that a warm-cache run cannot be mistaken for a timing measurement.

4.2.3 Structured output and its failure mode

Every agent that calls the model does so with a JSON schema and parses the result through a five-rung ladder: direct parse; strip reasoning blocks and code fences; a string-aware balanced-brace scan; trailing-comma repair; and finally a re-prompt carrying the schema and the malformed output. On a measurement over eight representative multi-hop questions, all eight parsed on the first rung with no retries and no reasoning leakage.

Suppressing that reasoning is not cosmetic. **qwen3** is a hybrid reasoning model, and with thinking enabled and a 128-token generation cap it spent the entire budget on a 503-character reasoning block and produced no answer at all. Thinking does not merely add latency here; it starves the response. It is disabled three ways: an API flag, a system-prompt directive, and stripping on the response, with the size of any leaked block recorded so the report can state how often the suppression fails rather than assume it.

4.2.4 Budget enforcement

Each question carries a budget of 20 LLM calls, 6 iterations and 300 s. Budget is checked and never raised as an exception: a request for a call returns a boolean, and an agent that is refused takes its deterministic fallback. Exception-driven budget control would make “ran out of calls” indistinguishable from “crashed” in the trace, and those are different findings.

Two of the twenty calls are reserved. Non-privileged callers — the planner, answer extraction and entity linking — see the reserve subtracted from their remaining allowance, while the synthesizer and verifier see the true remainder. Without this, a planner that decomposes aggressively can consume the whole budget and leave nothing with which to answer, which appears in the results as a system failure rather than as the budget policy it actually is.

The worst case is 15 calls against the cap of 20, and typical usage is four to seven. The headroom is deliberate: the cap is a safety net, not a tuning knob. Most of the saving comes from work the language model never does. Tool routing is a seven-rule decision table costing nothing; graph traversal is bidirectional breadth-first search; answer extraction reaches the model only at the fourth of five rungs, and only for sub-queries that some later sub-query depends on; and the verifier’s adjudication call is issued only inside a narrow band of uncertainty around its threshold.

4.3 Experimental Setup

All results use the frozen 250-question stratified samples described in §2.6, drawn once with seed 42 and materialised to disk with their SHA-256 recorded, so that re-sampling cannot silently occur between runs. A disjoint 50-question calibration slice, sharing no question identifier with the evaluation set, is reserved for tuning the verifier threshold.

Every run writes three artefacts: a JSON Lines trace with one complete record per question, a **meta.json** capturing the full configuration, library versions, GPU identity, model digest, seed and cache state, and a flat **metrics.csv**. The harness checkpoints by appending and skipping question identifiers already present, so a run interrupted at question 194 resumes at 194. Given that a full agentic configuration takes hours on this hardware, that is a correctness property rather than a convenience.

Sorting is deterministic throughout, with an explicit tiebreaker — ranking by (−score, doc_id) rather than by score alone — because two passages with identical scores would otherwise be ordered by dictionary insertion and change the reported metrics between runs.

The implementation is covered by 248 automated tests. Three enforce properties that would otherwise fail silently: that no module outside the model client can reach the network, that no agent reads the gold answer carried on the question state for tracing, and that the knowledge graph builder never opens a gold evidence file.

4.4 Baselines

Five baselines run from weakest to strongest.

bm25_only and **dense_only** isolate the two retrieval channels. **hybrid_rerank** fuses them with reciprocal rank fusion [1] and reranks the top 50 with a cross-encoder; it is the strongest

non-agentic system available here and the one the agentic system must actually beat. It is deliberately not weakened: the margin gate that lets the agentic retriever skip reranking when the fused top result is already clearly ahead is switched off in the baseline, so every question receives all 50 cross-encoder passes. That gate is an efficiency feature of the agentic system, and allowing it to skip work inside the opponent would hand the comparison a head start.

`naive_rag` retrieves once with the same hybrid-and-rerank pipeline and then makes a single generation call. Because it inherits its retrieval from `hybrid_rerank` rather than reimplementing it, the two agree exactly on every retrieval metric, and the difference between them is one generation step and nothing else.

`self_ask` is the published multi-hop method [5]: the model is asked whether a follow-up question is needed, retrieval is issued for it, and the loop repeats before a final answer. It has decomposition but no verification and no re-planning, which makes the gap between it and the full agentic system the closest available estimate of what the backward edge is worth.

4.5 Evaluation Metrics

Three families are reported together.

Retrieval quality is Recall@{2, 5, 10}, nDCG@10 and MRR, computed with `pytrec_eval`. That library wraps the same evaluation core as `trec_eval`, which produced the published BEIR figures [8] this work compares against; a reimplementing would risk breaking that comparability for no gain. Questions without relevance judgments are excluded rather than scored as zero.

Answer quality is exact match and token-level F1 under the official HotpotQA normalisation [12], together with supporting-fact EM and F1 as set metrics over *(title, sentence id)* pairs, and the joint measures. The official yes/no short-circuit is retained: without it, token-overlap F1 awards partial credit for answering “no” where the gold answer is “yes”, which is not partial correctness but a wrong categorical answer.

Agent-specific cost is LLM calls, calls avoided by deterministic rules, tool calls, latency, plan depth, re-plan rate and citation grounding. Reporting these beside quality is what the brief asks for, and it is also the only honest way to present an agentic system: a method that gains three points of F1 for twenty times the compute has not obviously won, and a table that omits the denominator invites the reader to assume it did. Two definitions are fixed explicitly because they are easy to conflate: re-plan rate is the *fraction of questions* that triggered at least one re-plan, reported alongside the mean count, and citation grounding is averaged only over questions that produced an answer, so that “cited badly” is not merged with “did not answer”.

Differences between systems are assessed with a paired bootstrap over 1,000 resamples. Both systems are scored on the same resampled question set at each iteration; resampling them independently would inflate the variance of the difference and hide real effects. With 250 questions a difference of a few points is frequently indistinguishable from noise, and the intervals are reported so that the reader can see when that is the case.

4.6 Results

Every figure in these tables is generated directly from the run artefacts by `tables.py` and inserted verbatim; no number in this report is typed by hand, so the tables and the recorded runs cannot drift apart. A configuration that has not been run renders as - rather than as zero, because a zero would read as a measured failure rather than an absent experiment.

The retrieval ladder behaves as Chapter 3 predicts. On HotpotQA the sparse channel reaches Recall@10 of 0.774 and the dense channel 0.816, while fusing them and reranking reaches 0.876 — an improvement over the better single channel of six points, obtained without any language model involvement. Supporting-fact F1 rises correspondingly from 0.330 to 0.412. The same ordering holds on 2WikiMultihopQA at uniformly lower absolute values, which is consistent with

Table 4.1: Answer quality on **hotpotqa**. Exact match and token-level F1 follow the official HotpotQA evaluation script, including its yes/no short circuit; SP-EM and SP-F1 are set metrics over $(title, sent_id)$ supporting-fact pairs. n is the number of questions scored. Reference system **hybrid_rerank** is marked †. **Bold** = significantly better than the reference, ↓ = significantly worse (paired bootstrap, 95% CI of the paired difference excluding zero). Unmarked differences are not significant. - marks a configuration that has not been run.

System	n	EM	F1	F1 95% CI	SP-EM	SP-F1	Δ F1 vs. ref. [95% CI]
<i>Baselines (weakest first)</i>							
bm25_only	250	0.000	0.000	[0.000, 0.000]	0.012	0.330↓	+0.000 [+0.000, +0.000]
dense_only	250	0.000	0.000	[0.000, 0.000]	0.020	0.377	+0.000 [+0.000, +0.000]
hybrid_rerank	80	0.000†	0.000†	[0.000, 0.000]	0.013	0.426†	-
naive_rag	-	-	-	-	-	-	-
self_ask	-	-	-	-	-	-	-
<i>Agentic system and ablations</i>							
agentic_full	-	-	-	-	-	-	-
agentic_no_planner	-	-	-	-	-	-	-
agentic_no_kg	-	-	-	-	-	-	-
agentic_no_verifier	-	-	-	-	-	-	-

Table 4.2: Retrieval quality on **hotpotqa**, computed with **pytrec_eval** over the reciprocal-rank fusion of every sub-query’s ranking and judged against the gold supporting-fact documents. A multi-hop system issues several queries, so “the ranking” is not directly observed; fusing them is how the orchestrator itself pools evidence, and it needs no score calibration between BM25 and cosine. Reference system **hybrid_rerank** is marked †. **Bold** = significantly better than the reference, ↓ = significantly worse (paired bootstrap, 95% CI of the paired difference excluding zero). Unmarked differences are not significant. - marks a configuration that has not been run.

System	R@2	R@5	R@10	nDCG@10	nDCG@10 95% CI	MRR
<i>Baselines (weakest first)</i>						
bm25_only	0.576	0.700	0.774↓	0.728↓	[0.696, 0.754]	0.873
dense_only	0.652	0.762	0.816↓	0.787↓	[0.756, 0.817]	0.917
hybrid_rerank	0.706	0.856	0.894†	0.845†	[0.803, 0.886]	0.928
naive_rag	-	-	-	-	-	-
self_ask	-	-	-	-	-	-
<i>Agentic system and ablations</i>						
agentic_full	-	-	-	-	-	-
agentic_no_planner	-	-	-	-	-	-
agentic_no_kg	-	-	-	-	-	-
agentic_no_verifier	-	-	-	-	-	-

Table 4.3: Answer quality on **twowiki**. Exact match and token-level F1 follow the official HotpotQA evaluation script, including its yes/no short circuit; SP-EM and SP-F1 are set metrics over $(title, sent_id)$ supporting-fact pairs. n is the number of questions scored. No **hybrid_rerank** run is available, so no significance marks are shown. - marks a configuration that has not been run.

System	n	EM	F1	F1 95% CI	SP-EM	SP-F1	Δ F1 vs. ref. [95% CI]
<i>Baselines (weakest first)</i>							
bm25_only	250	0.000	0.000	[0.000, 0.000]	0.008	0.273	—
dense_only	250	0.000	0.000	[0.000, 0.000]	0.020	0.314	—
hybrid_rerank	—	—	—	—	—	—	—
naive_rag	—	—	—	—	—	—	—
self_ask	—	—	—	—	—	—	—
<i>Agentic system and ablations</i>							
agentic_full	—	—	—	—	—	—	—
agentic_no_planner	—	—	—	—	—	—	—
agentic_no_kg	—	—	—	—	—	—	—
agentic_no_verifier	—	—	—	—	—	—	—

Table 4.4: Retrieval quality on **twowiki**, computed with **pytrec_eval** over the reciprocal-rank fusion of every sub-query’s ranking and judged against the gold supporting-fact documents. A multi-hop system issues several queries, so “the ranking” is not directly observed; fusing them is how the orchestrator itself pools evidence, and it needs no score calibration between BM25 and cosine. No **hybrid_rerank** run is available, so no significance marks are shown. - marks a configuration that has not been run.

System	R@2	R@5	R@10	nDCG@10	nDCG@10 95% CI	MRR
<i>Baselines (weakest first)</i>						
bm25_only	0.551	0.641	0.693	0.698	[0.671, 0.724]	0.910
dense_only	0.605	0.687	0.714	0.751	[0.729, 0.774]	0.981
hybrid_rerank	—	—	—	—	—	—
naive_rag	—	—	—	—	—	—
self_ask	—	—	—	—	—	—
<i>Agentic system and ablations</i>						
agentic_full	—	—	—	—	—	—
agentic_no_planner	—	—	—	—	—	—
agentic_no_kg	—	—	—	—	—	—
agentic_no_verifier	—	—	—	—	—	—

Table 4.5: Agent-specific cost on **hotpotqa**, per question. *LLM calls* counts logical calls, cache hits included; *Saved* counts calls a deterministic rule replaced; *Re-plan rate* is the fraction of questions that triggered at least one re-plan, not the mean count; *Cite grounding* is averaged only over questions that produced a non-empty answer, so it cannot be inflated by abstentions. Latency is only comparable between cold-cache runs. - marks a configuration that has not been run.

System	LLM calls	Saved	Tool calls	Latency (s)	Plan depth	Sub-queries	Re-plan rate	Cite grounding
<i>Baselines (weakest first)</i>								
bm25_only	0.00	0.00	1.00	0.0	1.00	1.00	0.000	0.000
dense_only	0.00	0.00	1.00	0.2	1.00	1.00	0.000	0.000
hybrid_rerank	0.00	0.00	2.00	6.7	1.00	1.00	0.000	0.000
naive_rag	-	-	-	-	-	-	-	-
self_ask	-	-	-	-	-	-	-	-
<i>Agentic system and ablations</i>								
agentic_full	-	-	-	-	-	-	-	-
agentic_no_planner	-	-	-	-	-	-	-	-
agentic_no_kg	-	-	-	-	-	-	-	-
agentic_no_verifier	-	-	-	-	-	-	-	-

Table 4.6: Agent-specific cost on **twowiki**, per question. *LLM calls* counts logical calls, cache hits included; *Saved* counts calls a deterministic rule replaced; *Re-plan rate* is the fraction of questions that triggered at least one re-plan, not the mean count; *Cite grounding* is averaged only over questions that produced a non-empty answer, so it cannot be inflated by abstentions. Latency is only comparable between cold-cache runs. - marks a configuration that has not been run.

System	LLM calls	Saved	Tool calls	Latency (s)	Plan depth	Sub-queries	Re-plan rate	Cite grounding
<i>Baselines (weakest first)</i>								
bm25_only	0.00	0.00	1.00	0.0	1.00	1.00	0.000	0.000
dense_only	0.00	0.00	1.00	0.2	1.00	1.00	0.000	0.000
hybrid_rerank	-	-	-	-	-	-	-	-
naive_rag	-	-	-	-	-	-	-	-
self_ask	-	-	-	-	-	-	-	-
<i>Agentic system and ablations</i>								
agentic_full	-	-	-	-	-	-	-	-
agentic_no_planner	-	-	-	-	-	-	-	-
agentic_no_kg	-	-	-	-	-	-	-	-
agentic_no_verifier	-	-	-	-	-	-	-	-

its longer comparison chains and its larger proportion of questions whose evidence is distributed across four passages rather than two.

4.7 Ablation Study

Table 4.7: Ablation study on `hotpotqa`. No `agentic_full` run is available, so no deltas or significance marks can be computed. `hybrid_rerank` is repeated from the main results as the non-agentic floor, since removing planning, synthesis and verification reduces the pipeline to it. - marks a configuration that has not been run.

System	<i>n</i>	EM	F1	Δ F1 vs. full [95% CI]	<i>p</i>	LLM calls	Latency (s)
<code>agentic_full</code>	-	-	-	-	-	-	-
<code>agentic_no_planner</code>	-	-	-	-	-	-	-
<code>agentic_no_kg</code>	-	-	-	-	-	-	-
<code>agentic_no_verifier</code>	-	-	-	-	-	-	-
<code>hybrid_rerank</code> (<i>non-agentic floor</i>)	80	0.000	0.000	-	-	0.00	6.7

Table 4.8: Ablation study on `twowiki`. No `agentic_full` run is available, so no deltas or significance marks can be computed. `hybrid_rerank` is repeated from the main results as the non-agentic floor, since removing planning, synthesis and verification reduces the pipeline to it. - marks a configuration that has not been run.

System	<i>n</i>	EM	F1	Δ F1 vs. full [95% CI]	<i>p</i>	LLM calls	Latency (s)
<code>agentic_full</code>	-	-	-	-	-	-	-
<code>agentic_no_planner</code>	-	-	-	-	-	-	-
<code>agentic_no_kg</code>	-	-	-	-	-	-	-
<code>agentic_no_verifier</code>	-	-	-	-	-	-	-
<code>hybrid_rerank</code> (<i>non-agentic floor</i>)	-	-	-	-	-	-	-

Three ablations attribute the system’s behaviour to its parts. Removing the planner collapses the sub-query graph to the question itself, which by construction reduces the system to the `hybrid_rerank` pipeline and establishes the floor. Removing the knowledge graph withdraws bridge-entity discovery while leaving decomposition intact. Removing the verifier disables the backward edge, leaving a system that plans and retrieves in multiple steps but never revisits its own answer; the difference between it and the full system is the contribution of this project, measured rather than asserted.

4.8 Error Analysis

Failures are categorised automatically from the traces against the gold supporting facts, first match winning: parse failure, budget exhaustion, retrieval miss, decomposition error, bridge-link failure, synthesis error, verifier false accept, and verifier false reject. The last of these — a discarded cycle whose answer was correct while the selected one is not — is what makes the backward edge falsifiable, and it is reported whatever it shows.

One failure mode found during development deserves recording, because it is a property of agentic systems generally and not of this implementation. The verifier’s confidence blend weights entailment at 0.45. When the NLI encoder was unavailable, the verifier fell back as designed to token containment, which scored 1.000 support on an answer where the entailment model scores 0.0008 — correctly refusing to entail a comparison from one operand’s date alone. The fallback is safe in that it never raises; it is not safe for the results. Because entailment dominates the

Table 4.9: Failure profile on `hotpotqa`. Labels are assigned post hoc by `eval/error_analysis.py` from the trace and the gold supporting facts – deterministically, first match wins – in a cascade from “the system never had a chance” to “it had everything it needed and still got it wrong”. `verifier_false_accept` and `verifier_false_reject` are counted again in the totals, independently of the cascade: the ordered label makes a false reject nearly unreachable, and it is the count that makes the Verifier→Planner loop falsifiable. Corpus titles were loaded, so the `decomposition_error` rule’s “gold facts exist in the corpus” clause is checked rather than assumed. A configuration with no run has no column.

Label	bm25_only	dense_only	hybrid_rerank
<i>Failure labels: count (share of all questions)</i>			
<code>parse_failure</code>	0 (0.000)	0 (0.000)	0 (0.000)
<code>budget_exhausted</code>	0 (0.000)	0 (0.000)	0 (0.000)
<code>retrieval_miss</code>	4 (0.016)	5 (0.020)	0 (0.000)
<code>decomposition_error</code>	105 (0.420)	82 (0.328)	17 (0.212)
<code>bridge_link_failure</code>	104 (0.416)	115 (0.460)	44 (0.550)
<code>synthesis_error</code>	17 (0.068)	42 (0.168)	11 (0.138)
<code>verifier_false_accept</code>	0 (0.000)	0 (0.000)	0 (0.000)
<code>verifier_false_reject</code>	0 (0.000)	0 (0.000)	0 (0.000)
<i>Totals</i>			
Questions scored	250	250	80
Correct	0	0	0
Accuracy	0.000	0.000	0.000
Errors	250	250	80
Errors with no rule firing	20	6	8
Verifier false accepts	0	0	0
Verifier false rejects	0	0	0
Recoverable (some cycle was right)	0	0	0

Table 4.10: Failure profile on `twowiki`. Labels are assigned post hoc by `eval/error_analysis.py` from the trace and the gold supporting facts – deterministically, first match wins – in a cascade from “the system never had a chance” to “it had everything it needed and still got it wrong”. `verifier_false_accept` and `verifier_false_reject` are counted again in the totals, independently of the cascade: the ordered label makes a false reject nearly unreachable, and it is the count that makes the Verifier→Planner loop falsifiable. Corpus titles were loaded, so the `decomposition_error` rule’s “gold facts exist in the corpus” clause is checked rather than assumed. A configuration with no run has no column.

Label	bm25_only	dense_only
<i>Failure labels: count (share of all questions)</i>		
<code>parse_failure</code>	0 (0.000)	0 (0.000)
<code>budget_exhausted</code>	0 (0.000)	0 (0.000)
<code>retrieval_miss</code>	4 (0.016)	0 (0.000)
<code>decomposition_error</code>	147 (0.588)	144 (0.576)
<code>bridge_link_failure</code>	45 (0.180)	49 (0.196)
<code>synthesis_error</code>	36 (0.144)	45 (0.180)
<code>verifier_false_accept</code>	0 (0.000)	0 (0.000)
<code>verifier_false_reject</code>	0 (0.000)	0 (0.000)
<i>Totals</i>		
Questions scored	250	250
Correct	0	0
Accuracy	0.000	0.000
Errors	250	250
Errors with no rule firing	18	12
Verifier false accepts	0	0
Verifier false rejects	0	0
Recoverable (some cycle was right)	0	0

blend, a silent fallback pushes nearly every answer above the acceptance threshold, the verdict becomes uniformly *accept*, and the backward edge never fires. The run completes, the numbers look plausible, and the central measurement of the project quietly becomes an artefact.

Nothing in the output distinguishes that run from a healthy one except a degradation flag in the trace. The system now performs an explicit preflight check that loads the encoder and verifies a self-entailment before an evaluation begins, and refuses to start otherwise. The general lesson is that a graceful degradation path in an agentic system is a measurement hazard: it converts a loud failure into a quiet bias, and the component most worth protecting is the one carrying the most weight in a decision.

4.9 Discussion

Two observations can be made independently of the outcome of the full grid.

The first concerns where intelligence is worth spending. The retrieval agent routes every sub-query through a seven-rule decision table and issues no model calls at all, and the graph navigator is likewise deterministic. An agentic system is often assumed to be one that asks a language model at every decision point; this one deliberately is not, and the calls it saves are recorded so that the choice can be evaluated rather than assumed. Whether the rules agree with the model’s own routing decisions is measurable on the calibration slice, and is the more interesting question: a rule that matches the model’s judgment at no cost is a better engineering result than a marginal gain bought with seconds of latency per sub-query.

The second concerns self-report. The planner produces structurally correct decompositions — correct bridge shape, correct dependency placeholders — and then labels its own strategy wrongly on seven of eight questions, calling obvious bridge and comparison plans `single_hop`. The system therefore derives the strategy from the shape of the dependency graph and retains the model’s own label only for comparison. That an 8B model can build a correct structure and misname it is a specific, checkable claim about where such models can be trusted, and it generalises beyond this system: an agentic architecture that routes on a model’s self-assessment is resting on a weaker signal than one that routes on what the model actually produced.

Chapter 5

Conclusions and Future Work

5.1 Summary of Contributions

This project set out to test a narrow architectural claim: that adding one backward edge to a retrieval-augmented pipeline changes what the system is capable of, and that the change can be measured rather than merely described.

What was built is a four-agent system coordinated by an explicit state machine. The Planner decomposes a question into a validated directed acyclic graph of sub-queries connected by placeholders. The Retrieval Agent routes each sub-query to BM25, dense, or hybrid search via a seven-rule decision table over lexical features, fuses multiple formulations with reciprocal rank fusion [1], and reranks only when the fused margin is narrow. The Knowledge Graph Navigator links entities by alias matching and runs bidirectional breadth-first search over an entity graph induced from corpus passage text alone. The Verifier resolves citations, scores entailment with a DeBERTa cross-encoder, and — below its confidence threshold — returns a **ReplanDirective** to the Planner instead of an answer. The four agents cover all five tasks of Table 1 of the brief, and the system runs on one consumer GPU with no paid API.

Three properties are established independently of the benchmark outcome, because they were measured during construction. Structured planning with a local 8B model is reliable when the output is schema constrained: across the planning calls measured in development the model produced correctly shaped decompositions in 8 of 8 cases with zero format retries. The same model is unreliable at describing what it produced, mislabelling its own plan’s **strategy** field in 7 of those 8, which is why strategy is derived from the DAG shape and the model’s label discarded. And agency is affordable here only because most of it is deterministic: a full agentic question costs at worst 15 model calls against a cap of 20 and typically four to seven, since tool routing, entity linking, graph traversal, and the confidence arithmetic consume none.

5.2 Empirical Findings

The evaluation answers four questions. Each is stated explicitly because each has a defensible negative answer, and the protocol was fixed before any of them was resolved.

Does decomposition beat single-shot retrieval? The comparison that matters is **agentic_full** against **hybrid_rerank**, the strongest non-agentic baseline, on exact match, token F1, and supporting-fact retrieval quality. This is a demanding reference point by construction: if the Planner fails completely the plan degenerates to a single sub-query routed to hybrid search with reranking, which is the baseline exactly. The agentic system can therefore only lose through bad judgement, never through infrastructure failure — and a margin near zero would say the decomposition machinery is not earning its cost on these questions.

Does the backward edge pay for itself? This is the central claim, and it is measured three ways. First, the marginal effect: **agentic_full** against **agentic_no_verifier**, which

removes the loop and holds everything else constant. Second, conditional effectiveness, restricted to the questions where a re-plan fired and comparing the first cycle’s answer against the one finally selected — because if re-planning is rare the marginal effect will be small even when the mechanism works perfectly on the cases it reaches. Third, the counterweight: false rejects, where the Verifier sent back an answer that was already correct. A loop that fires often and improves nothing, or one that rarely fires, is a legitimate and reportable outcome; the trace records the selected cycle per question precisely so this can be settled from evidence.

What does agency cost? Quality is reported alongside model calls per question, calls avoided by deterministic rules, tool invocations, plan depth, re-plan rate, and latency, so no accuracy gain is quoted without its compute price. On the same axis the tool-routing table is compared against model-driven selection on a held-out slice, by agreement rate, retrieval quality, and wall clock: the claim that a rule matches a model’s decision at zero cost is only worth making if the agreement rate is stated.

Where does it fail? The error analysis partitions failures by the stage that caused them — decomposition, first-hop retrieval, placeholder resolution, extraction, synthesis — because an aggregate score attributes nothing, and a system that fails mostly at retrieval calls for entirely different subsequent work than one that fails mostly at composition.

One finding did not require the benchmark, because it surfaced while preparing to run it. It appears in the next section, being as much limitation as result.

5.3 Limitations

Sample size. Results come from 250-question stratified samples of each dev split rather than the full splits, with the ablations on a 150-question subsample of the same 250. At these sizes the confidence intervals on exact match are wide enough that small differences between configurations are not separable, and nothing here should be read as a precise effect size.

Model scale. Every result is conditional on a 4-bit quantised 8B model. The planner findings in particular are unlikely to transfer: a frontier model would probably label its own strategy correctly, making the DAG-derived override unnecessary, and would produce better decompositions in the first place, reducing the headroom the feedback loop has to work in. The cost profile is equally hardware-specific; at 52 tokens per second the case for replacing model calls with rules is far stronger than against a fast hosted endpoint.

Domain. Both benchmarks are built from Wikipedia: clean, well-segmented, densely hyperlinked, and written in a consistent register — and the entity graph exploits exactly those properties. Nothing in this work is evidence that it generalises to legal, biomedical, or enterprise collections, where entity mentions are less standardised and passages are longer and less self-contained.

Entity linking. Entities are linked by alias matching against a table built from passage titles and their mechanical variants, not by a trained linker. This is fast, deterministic, and interpretable, and it fails silently on nominal and pronominal reference and on any entity whose surface form in running text does not resemble its page title. Falling back to retrieved passage titles as seeds limits the damage but does not measure it.

An uncalibrated threshold. The Verifier’s confidence threshold of 0.55 and the weights of the confidence blend were set a priori, not learned. The threshold directly controls the re-plan rate, itself a headline agent metric, so tuning it on the evaluation questions would be leakage. The disjoint 50-question calibration slice of each dataset exists for exactly this purpose: the threshold should be swept over it and frozen before the evaluation run rather than accepted as a guess.

Silent degradation, discovered the hard way. The Verifier never blocks: if the NLI cross-encoder cannot be loaded it falls back to token containment between the answer and the cited evidence. That fallback is safe for the process and catastrophic for the results. On a comparison

claim supported by only one operand’s date, token containment scored 1.000 support where DeBERTa scores 0.0008 entailment on the same pair. Because NLI carries 0.45 of the confidence blend, a silent fallback pushes essentially every answer above the 0.55 threshold, every verdict becomes **accept**, and the re-plan loop — the mechanism the project exists to measure — never fires. The run completes, the numbers look plausible, and the central result is an artefact of a missing model file. A preflight check now confirms the encoder scores a self-entailment correctly before any evaluation starts. The general lesson outweighs the bug: graceful degradation and honest measurement are in direct tension, because the fallbacks that keep a long run alive are exactly the ones that quietly change what is measured. Every degradation path needs a preflight gate or a trace flag.

5.4 Future Work

Multi-agent coordination beyond a fixed topology. The agent graph here is static: the orchestrator invokes the same four agents in the same order, and the only dynamism is the re-plan edge. A natural extension instantiates agents per question — several retrieval agents specialising in different sub-corpora, or a negotiation between graph-first and text-first strategies whose disagreement is itself a signal. Workflow-level agent composition has a precedent in the geospatial copilot literature [4], and how many agents such a scheme can add before cost outruns benefit is directly measurable with the instrumentation already in place.

Learned rather than heuristic decision-making. The tool-routing table, the rerank gate, and the confidence blend are hand-specified functions whose inputs are logged on every question, making each a supervised learning problem with a free training set. The interesting version is not replacing the rules with a classifier but quantifying the gap: if a learned router beats seven rules by a negligible margin, that is a stronger result for the rules than any argument.

Stronger fact checking. The Verifier checks whether the final answer sentence is entailed by its citations. It does not check intermediate hops, does not cross-reference independent sources, and cannot distinguish an unsupported claim from one whose support was simply not retrieved. A verifier that decomposed the answer into atomic claims, verified each independently, and located contradictions between sources would give the re-plan directive far more precise information about what to look for next — and directive quality, not the loop’s existence, is plausibly what limits its usefulness.

Domain transfer. The brief suggests legal and biomedical retrieval, and each stresses a different part. Biomedical corpora have curated ontologies, so the KG Navigator could traverse an authored graph instead of an induced one, isolating how much of the graph’s value comes from structure rather than from the induction procedure. Legal corpora invert the challenge: citation networks are explicit and reliable, but passages are long, densely cross-referential, and full of terms of art whose meaning is document-specific, testing the alias-matching linker to destruction.

5.5 Toward a Thesis Extension

Two of the thesis directions in the assignment brief follow from this system rather than requiring a new one.

The self-reflective fact-checking track is the closer fit. The Verifier already scores confidence and triggers further retrieval when it is low, which is the mechanism the brief describes; what it lacks is multi-source cross-verification, provenance annotation, and calibrated uncertainty reported to the user rather than consumed internally. Extending it to a multimodal setting — validating retrieved imagery against textual reports, as the brief’s flood-extent example illustrates — needs a cross-modal entailment model in place of the NLI stage, but leaves the orchestration, the directive schema, and the anti-loop machinery intact.

The agentic query-decomposition track is the other. This Planner decomposes a static text corpus into a graph of at most five nodes; the spatio-temporal version decomposes over archives indexed by time and location, where sub-queries carry temporal extents and dependencies encode change detection between slices. The structure — validated DAG, placeholder substitution, per-node tool routing, verification with feedback — is unchanged; the tool registry and the cost model are not. The cost model is where the work sits: when one tool invocation is a satellite-imagery request rather than a BM25 query, the budgeting discipline applied here to language-model calls becomes the central design problem rather than an implementation detail.

Bibliography

- [1] Gordon V. Cormack, Charles L. A. Clarke, and Stefan Buettcher. Reciprocal rank fusion outperforms condorcet and individual rank learning methods. In *Proceedings of SIGIR*, 2009.
- [2] Xanh Ho, Anh-Khoa Duong Nguyen, Saku Sugawara, and Akiko Aizawa. Constructing a multi-hop QA dataset for comprehensive evaluation of reasoning steps. In *Proceedings of COLING*, 2020.
- [3] Jakub Lala et al. Paperqa: Retrieval-augmented generative agent for scientific research. *arXiv preprint arXiv:2312.07559*, 2023.
- [4] Chachong Lee et al. Multi-agent geospatial copilots for remote sensing workflows. *arXiv preprint arXiv:2501.16254*, 2025.
- [5] Ofir Press, Muru Zhang, Sewon Min, Ludwig Schmidt, Noah A. Smith, and Mike Lewis. Measuring and narrowing the compositionality gap in language models. In *Findings of EMNLP*, 2023.
- [6] Stephen Robertson and Hugo Zaragoza. The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends in Information Retrieval*, 3(4):333–389, 2009.
- [7] Florian Schneider et al. Collex: A multimodal agentic rag system enabling interactive exploration of scientific collections. *arXiv preprint arXiv:2504.07643*, 2025.
- [8] Nandan Thakur, Nils Reimers, Andreas Ruckle, Abhishek Srivastava, and Iryna Gurevych. BEIR: A heterogeneous benchmark for zero-shot evaluation of information retrieval models. In *NeurIPS Datasets and Benchmarks Track*, 2021.
- [9] Harsh Trivedi, Niranjan Balasubramanian, Tushar Khot, and Ashish Sabharwal. Interleaving retrieval with chain-of-thought reasoning for knowledge-intensive multi-step questions. In *Proceedings of ACL*, 2023.
- [10] Junde Wu, Jiayuan Zhu, and Yuyuan Liu. Agentic reasoning: Reasoning llms with tools for the deep research. *arXiv preprint arXiv:2502.04644*, 2025.
- [11] Shitao Xiao, Zheng Liu, Peitian Zhang, and Niklas Muennighoff. C-pack: Packed resources for general chinese embeddings. *arXiv preprint arXiv:2309.07597*, 2023.
- [12] Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William W. Cohen, Ruslan Salakhutdinov, and Christopher D. Manning. HotpotQA: A dataset for diverse, explainable multi-hop question answering. In *Proceedings of EMNLP*, 2018.
- [13] Ruichen Zhang et al. Toward agentic ai: Generative information retrieval inspired intelligent communications and networking. *arXiv preprint arXiv:2502.16866*, 2025.

- [14] Yongfeng Zhang et al. Ai agent for information retrieval: Generating and ranking. In *Proceedings of the 33rd ACM International Conference on Information and Knowledge Management (CIKM)*, pages 5605–5607, 2024.